



UNIVERSITAT
POLITÈCNICA
DE VALÈNCIA

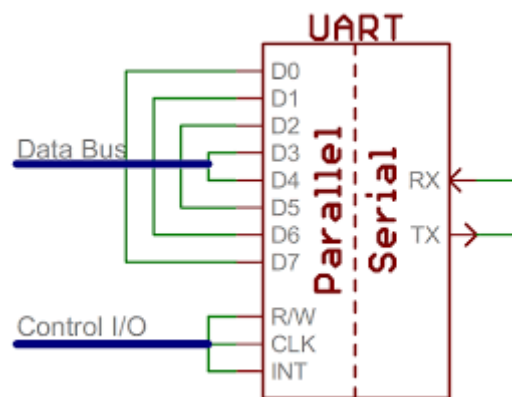
Grado en Informática Industrial y Robótica

Asignatura: Redes Industriales

Código de la asignatura: 14541

Práctica de Laboratorio

“Práctica 1. Comunicación Serie utilizando la
UART”



Profesor responsable:

Dr. Lenin G. Lemus Zúñiga

Página en Blanco

Contenido

Introducción:	7
Cadena de herramientas	Error! Bookmark not defined.
Headers del ESP32-S3-DevKitC-1	Error! Bookmark not defined.
Objetivo de la práctica	Error! Bookmark not defined.
Material necesario	Error! Bookmark not defined.
Desarrollo	Error! Bookmark not defined.
Creación y configuración del proyecto	Error! Bookmark not defined.
Ejemplo: GPIO	Error! Bookmark not defined.
Funciones GPIO :	Error! Bookmark not defined.
Prueba:	Error! Bookmark not defined.
Apéndice A.	Error! Bookmark not defined.

Página en blanco

Tabla de figuras

Figura 1.Comunicación entre UARTs.....	7
Figura 2. Señales para el control de flujo.	8

Página en blanco

Introducción:

La comunicación serie es un protocolo de transmisión de datos que envía información secuencialmente, bit a bit, a través de un solo cable o canal, a diferencia de la paralela que usa múltiples vías. Es fundamental en electrónica digital para conectar microcontroladores, sensores y computadoras, destacando por su simplicidad, uso de pocos pines y largas distancias.

El protocolo UART (Universal Asynchronous Receiver-Transmitter) es un protocolo de comunicación serie asíncrono, ampliamente utilizado para la comunicación entre microcontroladores, sensores, módulos Bluetooth y computadoras. Utiliza solo dos cables (TX y RX) para la transmisión bidireccional de datos bit a bit, sin requerir una señal de reloj compartida. Los dispositivos deben configurarse a la misma velocidad en baudios.

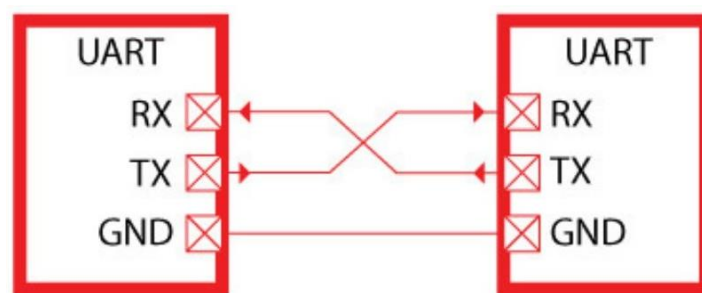


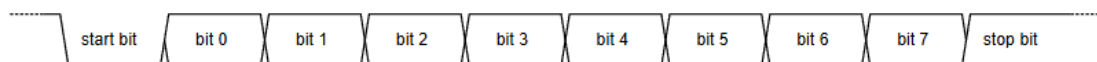
Figura 1. Comunicación entre UARTs.

Funcionamiento

El protocolo UART es un protocolo de comunicación de hardware P2P (peer-to-peer) en donde uno de los extremos puede ser un MCU (microcontrolador) y el otro extremo puede ser otro MCU, un sensor o un ordenador personal (a través de un convertidor USB a UART).

Atención: Una UART envía la información por BYTES, un BYTE en cada transmisión, y dado que la transmisión es asíncrona cuando una de las UART va a enviar el BYTE, utiliza un formato conocido como formato de BIT.

La transferencia de datos se realiza en serie. Comienza con un bit de inicio, generalmente activando la lógica a nivel bajo durante un ciclo de reloj. En los siguientes n ciclos de reloj, se envían n bits secuencialmente desde el transmisor (n puede ser 6, 7 o 8 en general suele ser 8). Opcionalmente, se puede añadir un bit de paridad para mejorar la fiabilidad de la transferencia. Finalmente, el cable de datos suele activarse para indicar el final de la transferencia. El bit de parada puede tener una duración de $\frac{1}{2}$ tiempo de bit, 1 tiempo de bit o dos tiempos de bit.



Atención: Las comunicaciones a través de la UART se especifican indicando la tasa de transmisión (9600 bits/segundo), el número de bits a transmitir (6, 7 y 8) el número de bits de parada y si utiliza un bit de paridad para determinar si hay errores en la transmisión. De forma que las características de la transmisión se especifican indicando la velocidad de transmisión en bits/seg, un dígito que indica el número de bits, una letra para indicar la paridad (N=Sin

paridad, E=paridad par ó O= paridad impar) y finalmente el número de bits de parada. Por ejemplo: 9600 8N1

Características Principales de la UART:

- Comunicación Asíncrona: No utiliza una señal de reloj común para sincronizar transmisor y receptor. La sincronización se logra mediante bits de inicio (start) y parada (stop) añadidos a cada trama de datos.
- Conexiones: Utiliza principalmente dos pines: TX (Transmisión) y RX (Recepción).
- Comunicación: Full-Duplex (ambos extremos pueden transmitir simultáneamente).
- Conexión entre dispositivos: El pin TX de un dispositivo se conecta al RX del otro, y viceversa, compartiendo la misma tierra (GND).
- Configuración: La velocidad de transmisión, conocida como baudios (ej. 9600, 115200), debe ser idéntica en ambos dispositivos.
- Niveles de Tensión: UART comúnmente opera a niveles TTL (0V - 3.3V o 5V), diferenciándose de RS-232, que utiliza voltajes más altos.
- Es Importante conectar las señales de masa.

Control de flujo:

Para que dos dispositivos puedan comunicarse es fundamental utilizar un protocolo, en este caso el Protocolo UART.

Para ello es necesario agregar las señales CTS (Clear To Send) y RTS (Ready To Send)

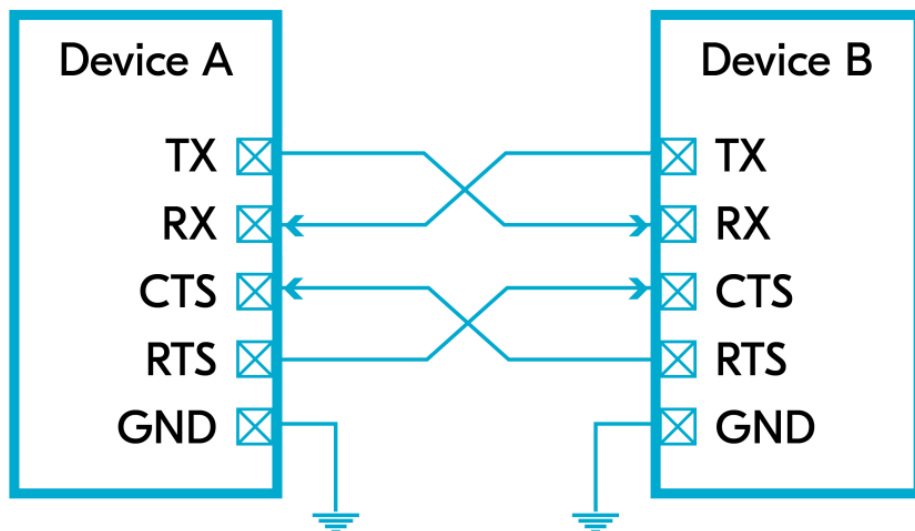
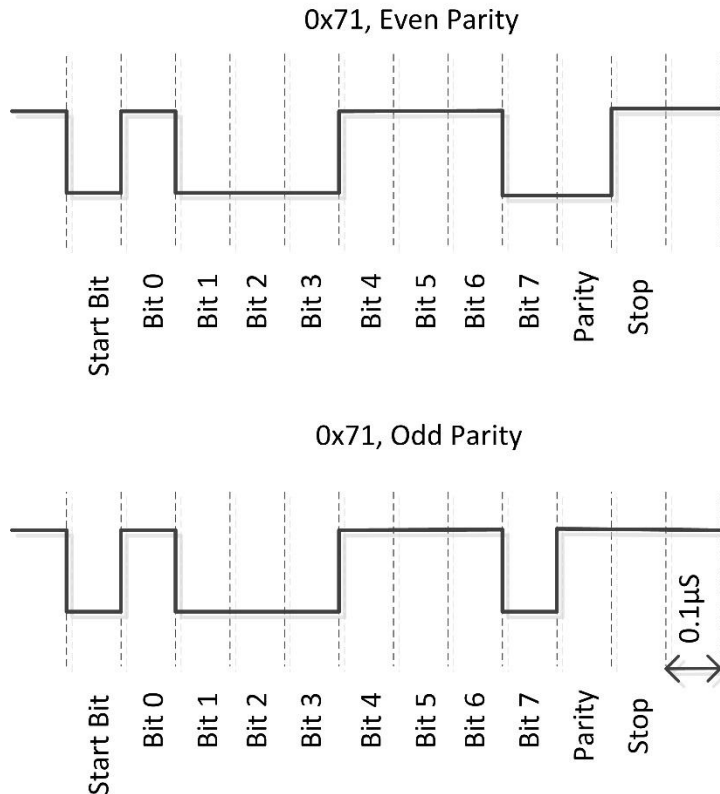


Figura 2. Señales para el control de flujo.

Por ejemplo, cuando la UART "Device A" está preparada para enviar información, activa la señal CTS, la UART "Device B" detecta que en la entrada RTS se ha activado, lo que significa que van a empezar a llegar los BYTES que contienen la información. Y acto seguido la UART "Device A"

inicia la transmisión de la información BYTE por BYTE. Cuando la UART “Device B” ha terminado de transmitir todos los datos de la información desactiva la señal CTS.

A continuación, se muestra el formato de cada BYTE transmitido, para una comunicación con paridad PAR



Pregunta: si el tiempo de bit es de 0.1 micro segundo. ¿Cuál es la tasa de transmisión?

Respuesta: _____Bits/seg

¿Cómo se especifica cada una de estas transmisiones?

Respuesta (paridad par): _____

Respuesta (paridad impar): _____

Escriba en formato binario, los bits que se envían. Respuesta: _____₂

Escriba en formato hexadecimal los bits que pasan a través del canal de comunicación:

Respuesta: 0x_____

Objetivo:

- Aprender a enviar y recibir datos a través de UART en modo asíncrono
- Aprender a configurar el hardware periférico UART mediante la API de UART.
- Examinar y practicar el uso de la API del controlador UART, para enviar datos de un PC a un microcontrolador

Material para el desarrollo de la práctica:

Hardware:

1 Ordenador Personal



1 Cable USB a TTL

1 Placa de desarrollo ESP32-S3



Software:

En el ordenador personal debe tener instalados los siguientes entornos de desarrollo:

- Arduino IDE
- Netbeans
- Driver para el cable USB a TTL

Interconexión de los componentes:

En el ESP32-s3

MCU	UART	RX GPIO	TX GPIO	Note
ESP32	UART0	3	1	Serial
	UART1	9	10	Serial1
	UART2	16	17	Serial2
ESP32-S2	UART0	44	43	Serial or Serial0 (*)
	UART1	18	17	Serial1
ESP32-S3	UART0	44	43	Serial or Serial0 (*)
	UART1	15	16	Serial1
	UART2	19	20	Serial2
ESP32-C3	UART0	20	21	Serial or Serial0 (*)
	UART1	18	19	Serial1

El Arduino Mega 2560 cuenta con 4 puertos serie UART por hardware (puertos serie TTL a 5V). Estos UART permiten la comunicación serie simultánea con múltiples dispositivos, además de la conexión USB para la programación, distribuidos en los siguientes pines:

- UART 0: Pines 0 (RX) y 1 (TX) - Usado también para USB.
- UART 1: Pines 19 (RX) y 18 (TX).
- UART 2: Pines 17 (RX) y 16 (TX).
- UART 3: Pines 15 (RX) y 14 (TX).

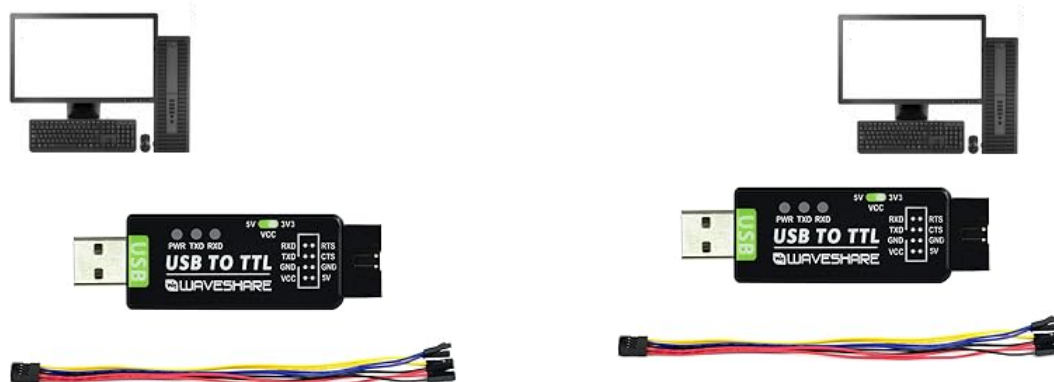
Desarrollo de la práctica:

Primera parte: Crear un programa que envíe datos a través de uno de los puertos COM.

Segunda parte: Visualización de las señales al transmitir el BYTE correspondiente al carácter 'A'

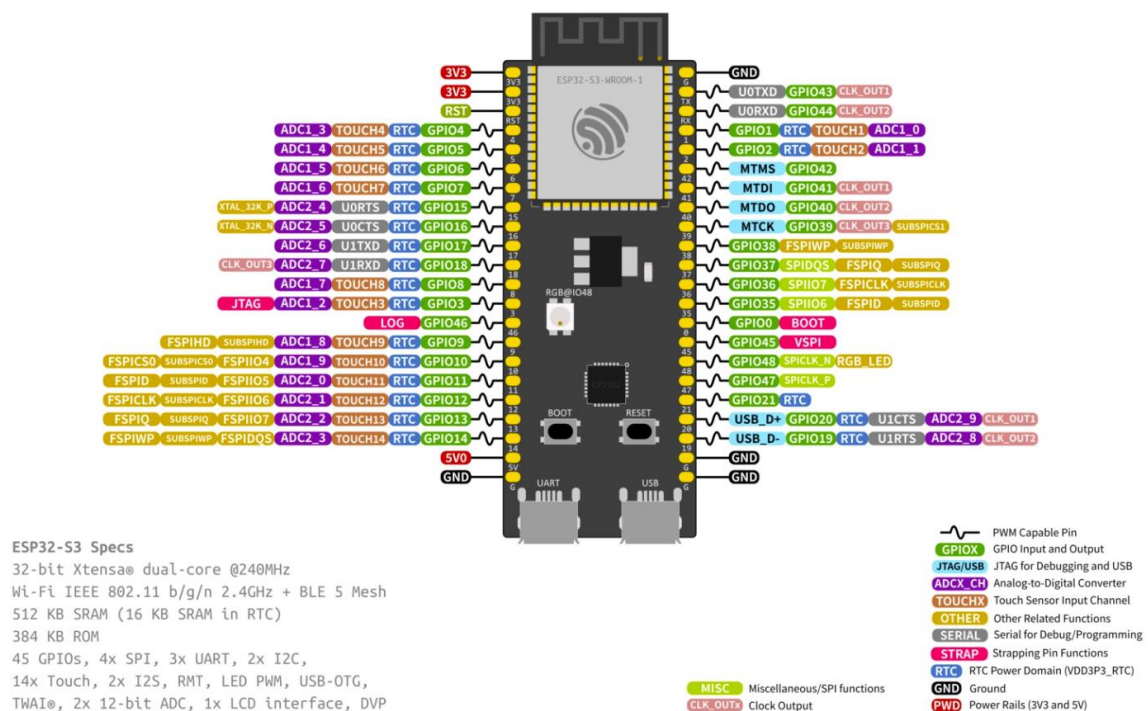
Ampliaciones

1. Transmitir un fichero entre dos ordenadores.



2. Transmitir un fichero entre un ordenador y una placa de desarrollo (ESP32-S3-DevKitC-1)

ESP32-S3-DevKitC-1



En la placa de desarrollo con la que se cuenta en el laboratorio (ESP32-S3-DevKitC-1) se aconseja utilizar la UART 1 (GPIO 17: U1TX, GPIO18:U1RX, GPIO19: U1RTS y GPIO20: U1CTS)

3. Transmitir un fichero entre dos placas de desarrollo (ESP32-S3-DevKitC-1)

Apéndice A.

El control de flujo por hardware UART es una técnica que permite que dispositivos con diferentes velocidades de procesamiento se comuniquen de forma fiable a través de una conexión UART sin pérdida de datos.

Por ejemplo, cuando el dispositivo A transmite un flujo continuo de bytes al dispositivo B, y este funciona a menor velocidad que el dispositivo A, el dispositivo B puede indicar al dispositivo A que pause temporalmente la transmisión, lo que le da tiempo para procesar los datos ya recibidos y liberar búferes de recepción. El control de flujo lo hace posible proporcionando señales adicionales que indican al dispositivo A que pause o reanude la transmisión de datos según sea necesario. Estas señales adicionales se denominan RTS (solicitud de envío) y CTS (autorización de envío).

Estas señales están interconectadas entre los dos dispositivos, donde el RTS de un dispositivo se conecta al CTS del otro, y viceversa. Cada dispositivo escribe su RTS para indicar que está listo para recibir nuevos datos, mientras que lee el CTS para determinar si puede enviar datos al dispositivo homólogo. Un dispositivo mantiene la línea RTS activada mientras esté preparado para aceptar datos adicionales y la desactiva poco antes de que su búfer de recepción alcance su capacidad máxima. El control de flujo por hardware funciona en ambas direcciones, lo que significa que cada dispositivo (Dispositivo A o Dispositivo B) puede indicar una pausa en la transmisión.

El periférico UART de los SoC Nordic admite el control automático de flujo por hardware, que se describe en detalle en la hoja de datos (especificaciones del producto) de cada SoC. En Periféricos -> UART -> Transmisión y Recepción.

Referencias Bibliográficas:

<https://academy.nordicsemi.com/courses/nrf-connect-sdk-fundamentals/lessons/lesson-4-serial-communication-uart/topic/uart-protocol/>